

THESIS PROPOSAL OUTLINE

CTU-SIGNBRIDGE: A MULTI-TENANT PLATFORM FOR VIETNAMESE SIGN LANGUAGE DATA COLLECTION AND MANAGEMENT

Sub-system: Core Data Collection & Management Infrastructure Layer

- **Student Name:** Mai Nhat Minh
- **Advisor 1:** Ph.D Truong Minh Thai - **Staff ID:** 000520

I. INTRODUCTION

1. Problem Statement / Rationale

- **Context of Sign Language Recognition (SLR)**

Sign language is one of the primary means of communication for deaf and hard-of-hearing individuals worldwide. In Vietnam, Vietnamese Sign Language (VSL) plays an important role in communication, education, and social inclusion within the deaf community. Recent advances in artificial intelligence and computer vision have accelerated the development of sign language technologies, including sign language recognition, translation, and educational support systems. The effectiveness of these technologies, however, depends heavily on the availability of high-quality, well-organized, and representative sign language datasets.

- **Challenges in Vietnamese Sign Language Dataset Development**

Compared with well-established sign language resources such as American Sign Language (ASL) and German Sign Language (DGS), Vietnamese Sign Language remains relatively under-resourced. Existing VSL datasets are commonly developed for individual research projects, typically containing limited vocabularies and independently defined annotation schemes. Furthermore, regional dialectal variations across different areas of Vietnam introduce additional complexity to dataset collection, annotation, and management. These factors reduce interoperability between datasets and limit their reuse in subsequent research.

- **Limitations of Existing Data Collection Approaches**

Current VSL data collection efforts are generally designed for individual research studies and commonly rely on general-purpose file storage services and manually organized directory structures. While suitable for small-scale data collection, these approaches provide limited support for standardized metadata management,

collaborative contribution, and long-term dataset maintenance. In addition, supporting multiple independent organizations or research groups within a shared software platform requires mechanisms for tenant isolation, access control, and resource management that are not readily provided by general-purpose storage solutions.

- **Proposed Solution**

To address these challenges, this research proposes CTU-SignBridge, a web-based multi-tenant platform for Vietnamese Sign Language dataset collection and management. The platform enables universities, research laboratories, special education institutions, community organizations, lecturers, students, and independent research groups to establish and manage their own VSL datasets within a shared software infrastructure while maintaining logical data isolation between tenants.

The proposed platform provides standardized dataset organization, project-based collaboration, identity and access management, browser-based landmark extraction, and asynchronous data processing to support efficient dataset collection and management. By providing a reusable software infrastructure for VSL dataset development, CTU-SignBridge aims to facilitate future research on Vietnamese Sign Language and contribute to the development of assistive technologies that improve accessibility and communication for the deaf and hard-of-hearing community.

2. Research Objectives

- **General Objective:**

To design, implement, and evaluate CTU-SignBridge, a web-based multi-tenant platform that supports the standardized collection, organization, and management of Vietnamese Sign Language datasets for multiple independent organizations and research groups.

- **Specific Objectives:**

- Design a multi-tenant software architecture that enables multiple organizations and research groups to manage independent VSL datasets within a shared platform while ensuring logical data isolation and secure access control.
- Develop a standardized dataset management model for organizing VSL vocabularies, regional dialects, metadata, contributors, and project resources.

- Implement a browser-based landmark extraction workflow to improve storage efficiency during the data collection process.
- Develop an asynchronous data processing mechanism to support scalable dataset ingestion, storage, and management.
- Evaluate the proposed platform in terms of functionality, data management capability, storage efficiency, system performance, and multi-tenant data isolation.

3. Scope and Delimitation

- **Research Subject (Target Object):**

This research focuses on the software architecture, multi-tenant data management model, database design, and data processing workflow of a web-based platform for Vietnamese Sign Language dataset collection and management.

- **Research Scope:**

This study focuses on the software engineering aspects of designing, implementing, and evaluating a web-based multi-tenant platform for Vietnamese Sign Language data collection and management. The research boundaries are defined as follows:

- **Included:**

- **Multi-Tenant Architecture:** Design and implementation of a Workspace–Project hierarchy that enables multiple independent organizations to operate within a shared platform while maintaining logical data isolation through row-level scoping. Each workspace functions as an independent tenant with its own projects, contributor lists, and dataset ownership.
- **Identity and Access Management:** Implementation of a token-based authentication system supporting user registration, login, password recovery, and session management. The system enforces role-based access control with multiple privilege levels (Owner, Admin, Contributor, Viewer) scoped at the workspace and project levels.
- **Standardized Vocabulary and Metadata Management:** Development of a relational taxonomy schema in PostgreSQL 16 that organizes sign vocabulary across four Vietnamese Sign Language regional dialects (Northern, Central, Southern, and Can Tho). The schema includes a taxonomy forking mechanism that allows tenants to extend shared vocabularies without corrupting the canonical dataset.
- **Browser-Based Landmark Extraction:** Integration of MediaPipe Holistic [8] on the client side to extract 42 hand landmarks (126

coordinate values per frame) directly in the browser. Extracted landmarks are stored as compressed NumPy arrays (.npz), reducing storage requirements by over 90% compared to raw video files.

- **Asynchronous Data Processing:** Implementation of a background task queue using Celery and Redis for video ingestion, sliding-window segmentation, data augmentation, and cloud synchronization with Google Drive. This architecture ensures that computationally expensive processing tasks do not block the main application.
- **Web Application Stack:** The backend is implemented using FastAPI (Python) with a layered architecture (Routers, Services, Processing, Storage, Workers). The frontend is built with React 18, TypeScript, Vite, and Tailwind CSS v4. PostgreSQL 16 serves as the primary relational database.
- **Deployment Environment:** All services are containerized and deployed using Docker Compose on a single-node environment. The deployment configuration includes the backend API server, frontend static files served via Nginx, Celery worker, Redis message broker, and PostgreSQL database.
- **Real-Time Recognition Demonstration:** A real-time sign recognition module with text-to-speech output is included as a functional demonstration of downstream dataset utility and as a communication aid for deaf users. This module operates on CPU-based inference within the backend service.

- **Excluded:**

- Training, optimization, or evaluation of downstream Sign Language Recognition (SLR) deep learning models. The platform provides dataset infrastructure; model development is outside the scope of this study.
- Research on new computer vision algorithms or landmark extraction methods. The study applies existing MediaPipe Holistic technology rather than proposing novel extraction techniques.
- Production-scale real-time sign language translation services. The recognition module serves as a technical demonstration, not a production-grade translation system.
- Large-scale distributed deployment and cloud-native orchestration. Horizontal auto-scaling, Kubernetes-based deployment, and high-availability configurations are reserved for future work.
- Full-body pose and facial expression capture. The database schema reserves fields for these features, but recording and processing pipelines for pose and facial data are not implemented in the current version.

II. THEORETICAL BACKGROUND / LITERATURE REVIEW

1. Core Concept 1 (e.g., Software Testing, Generative AI, etc.)

- **Definitions, Significance, and Standard Workflows:** ...
- **State-of-the-art Tools & Frameworks:** ...
- **Core References:**
 - Myers, G. J. (2011). *The Art of Software Testing*. Wiley.
 - IEEE Std 829-2008 – *Software and System Test Documentation*.
 -

2. Core Concept 2 / 3 (if applicable)

- (Add more theoretical sections depending on your thesis topic, e.g., Agentic Workflows, DevOps Pipeline integration, etc.)

III. METHODOLOGY

1. Research Approach

(Describe your scientific approach, design methodologies, such as Agile/Scrum, Model-Driven Development, or empirical evaluation methods.)

...

2. Implementation Roadmap / Work Plan

No.	Tasks / Activities	Expected Deliverables
1	Literature Review & Tool Survey	Comprehensive Review Report
2	System Design	Architectural Models, Database Schema, Algorithm Flowcharts
3	Development & Implementation	Executable Codebase (Frontend, Backend, APIs)
4	Testing & Verification	Test Plan, Test Cases, Execution Log & Test Reports
5	Evaluation, Thesis Writing & Refinement	Final Evaluation Report & Completed Thesis Draft

IV. PRACTICAL APPLICATIONS AND DATASETS

1. Real-World Applications

(Describe how your research outcomes or prototypes can be deployed or used in practical software engineering pipelines.)

...

2. Experimental Datasets

Dataset	Description	Source/Link
Defects4J	A database of real, reproducible software bugs	https://github.com/rjust/defects4j
CodeSearchNet	Large-scale dataset for multilingual code representation	https://github.com/github/CodeSearchNet
OpenTelemetry	Observability framework for system behavior data	https://opentelemetry.io

V. RELATED WORK

(The following table serves as an illustrative example. Students should customize it based on their specific research focus.)

Project / Repository	Description	Reference Link
TestPilot (Microsoft)	An AI-driven assistant for automated unit test generation	https://github.com/githubnext/testpilot
AwesomeLLM4UT	A curated collection of research papers on LLM for Unit Testing	https://github.com/iSEngLab/AwesomeLLM4UT

Project / Repository	Description	Reference Link
LangChain + CI/CD Demo	Integrating LLM agents into continuous testing pipelines	https://github.com/langchain-ai/chain

VI. EXPECTED OUTCOMES AND CONTRIBUTIONS

(The following are examples. Customize these points to match your project deliverables.)

1. A functional prototype of an Autonomous AI Agent capable of automated test case generation and execution.
2. A lightweight testing pipeline was integrated successfully into small-to-medium-scale software projects.
3. A detailed performance evaluation report assessing the efficiency and limitations of the proposed approach, with future work suggestions.
4. *(Optional)* A systematic guide/reference material for subsequent research on AI-assisted Software Engineering.

VII. REFERENCES

(Crucial Note for Students: You must manage your references using standard citation management tools such as **Mendeley** ([Download Link](#)) or **Zotero** ([Download Link](#)). Follow APA 7th style or IEEE format consistently.)

1. Winteringham, M., 2024. *Software testing with generative AI*. Simon and Schuster.
2. Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing*. Wiley.
3. Naik, K., & Tripathy, P. (2011). *Software Testing: Principles and Practices*. Pearson Education.
4. Zhang et al. (2025). *Large Language Models for Unit Testing: A Systematic Literature Review*. arXiv:2503.10521.
5. Kohl et al. (2026). *Automated Structural Testing of LLM-based Agents*. arXiv:2601.18827.
6. Chu, B., Feng, Y., Liu, K., Guo, Z., Zhang, Y., Shi, H., ... & Xu, B. (2025). Large Language Models for Unit Test Generation: Achievements, Challenges, and Opportunities. *arXiv preprint arXiv:2511.21382*.
7. LangChain Documentation – <https://python.langchain.com>.
8. n8n Documentation – <https://n8n.io>.
9. Defects4J Dataset – <https://github.com/rjust/defects4j>.

10. CodeSearchNet Dataset – <https://github.com/github/CodeSearchNet>.